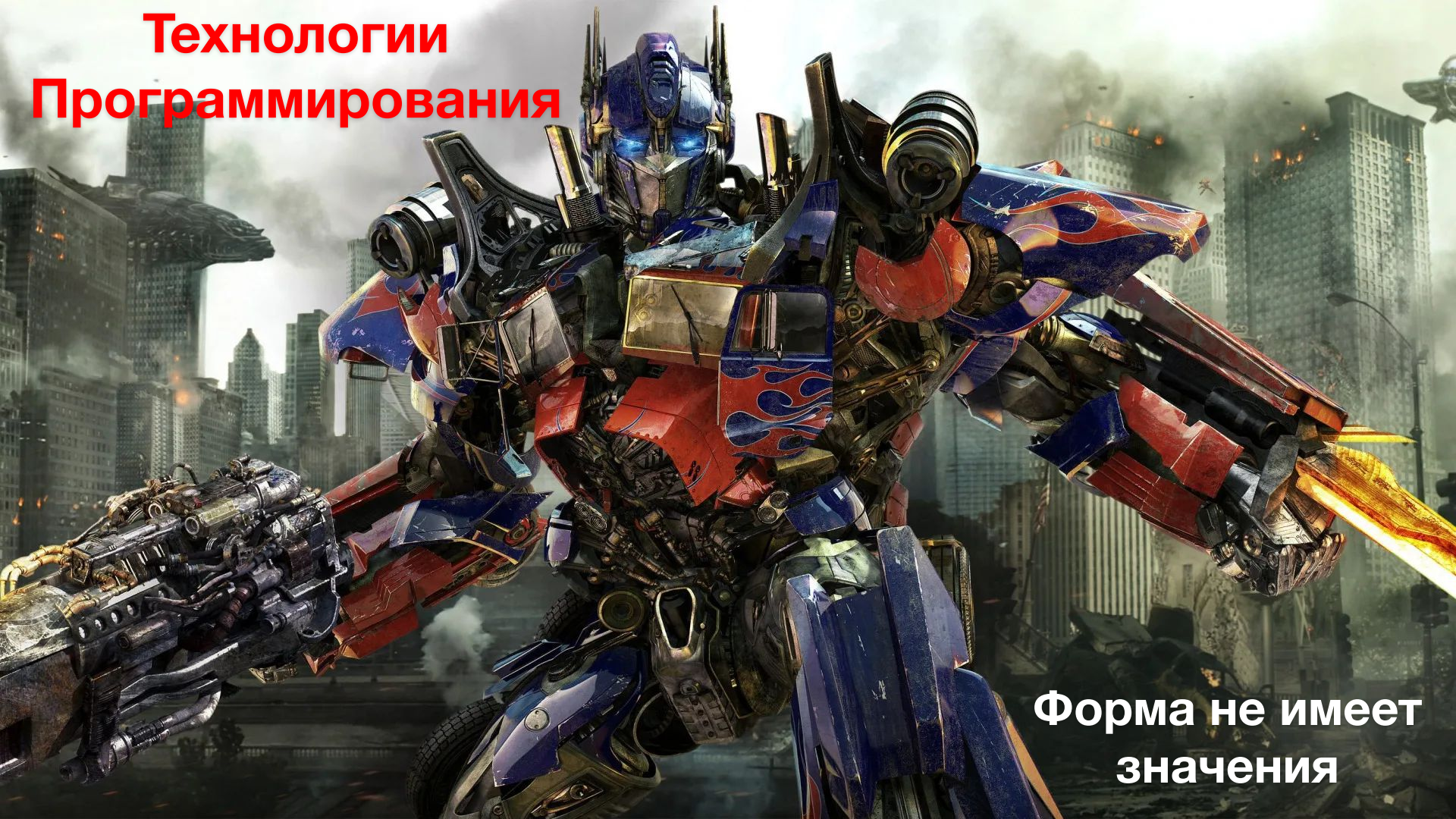


Технологии Программирования



Форма не имеет
значения

Вопрос

В двух словах, как код превращается в исполняемую программу?

Подсказка: в С и С++ этот процесс идентичен.

Препроцесс - Компиляция - Трансляция(асемблирование) - Линковка

Вопрос

Run-time VS Compile-time

Run-time - выполнение кода в момент работы программы
Compile-time - выполнение кода (задач) во время компиляции программы

Вопрос

Основная характеристика семейства языков C?

**Семейство языков C - языки со строгой типизацией:
От C - наиболее мягкая к C# - наиболее строгая.**

Наша форма неизменна

- Все поколение языков C - языки со сильной типизацией
 - Типы определяются в момент инстанциации переменных
 - Приведение типа не производится автоматически если оператор конверсии типа отсутствует
 - Нельзя совмещать разные типы в одном выражении
- Строго говоря, языки с сильной типизацией запрещают операцию приведения типа.



**Не будем забывать про то что
5 + 6 не всегда равно 11 в C++**

Раскол среди праймов

Оператор приведения типа

```
float floatParameter = 1.0f;
```

- **C-style приведение типа (type)variable**

- Старый формат,
- Небезопасен,
- Является источником многих ошибок несоответствия типов.

```
uint32_t integerParam = (uint32_t)floatParameter;
```

- **C++ стиль**

- **static_cast<type>(variable)** - приведение к типу type,
- **dynamic_cast<type>(pointer)** - приведение по цепочке наследования к указанному типу. Работает с указателями,
- **reinterpret_cast<type>(variable)** - аналог C-style cast только еще более опасный, приводит типы без каких либо проверок,
- **const_cast<type>(variable)** - делает из константы переменную.

```
uint32_t integerParam = static_cast<uint32_t>(floatParameter);
```



Автобот

auto - старый знакомый

- Заменяет собой тип,
- Выводит тип из контекста,
- Определяет тип переменной,
- Определяет тип возвращаемого значения из функции,

Плюсы

- Прост в использовании,
- Удобен,
- Краток,
- Почти нет минусов.

```
auto result = 5r + 6r;
```

Напоминаю про риск обфускации кода



Трансформер



Шаблон

```
UniversalMultifieldStruct<uint32_t, float, 4> templateStructure = {10, 2.f, {}};  
// имя шаблонного типа | параметры шаблона | объект шаблонного типа | инициализаторы
```

- Инструкция для компилятора по интерпретации типов данных,
- Шаблонный тип выводится (определяется) в момент использования, а не написания,
- Каждый вывод шаблона с отличающимися параметрами создает новый тип в вашей программе,
- Шаблоны существуют для типов,
- Шаблоны существуют для функции.

```
std::array<uint32_t, 16> array{0}; //uint32_t - тип параметров в массиве, 16 - его размер
```

```
uint32_t c_array[16] = {0}; // эквивалент, однако с ним не будут работать функции STL
```

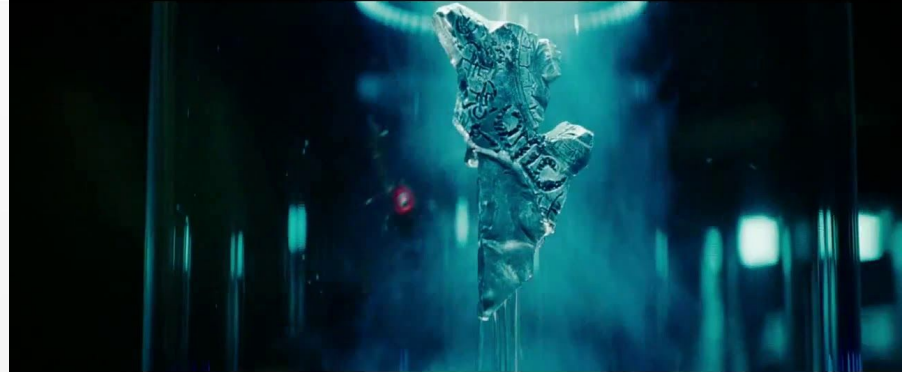

Искра

Вывод шаблона

- Шаблон выводится на этапе компиляции кода,
- Раскрутку шаблонов и их вычисление выполняется самим компилятором,

Метакод

- Сложный метакода может быть любой, но
 - Компилятор учитывает возможности метапрограммирования, однако не все Corner cases учтены,
 - Сложные конструкции могут обрушить компилятор (в лучшем случае),
 - Компилятор может собрать неожиданный код и получить совершенно неожиданный результат для программиста (в худшем случае),
 - Отлаживать мета-код практически невозможно.



Утопить компилятор в багах очень легко

Простой бот



Шаблоны типов

- **Для создания универсального типа используемого в программе**
 - Когда точный тип неизвестен или не универсален
 - Параметры типа неизвестны или не универсальны
- **Применяется для структур и классов**
 - Позволяет определять тип полей в момент создания структуры

```
template<typename T>
struct UniversalStruct
{
    T field;
};
UniversalStruct<uint32_t> intStruct;    // поле field здесь будет иметь тип uint32_t
UniversalStruct<float>    floatStruct; // поле field здесь будет иметь тип float
```

При инстанцировании одного шаблона с разными параметрами компилятор создает РАЗНЫЕ типы на каждый набор параметров

Прайм

Сложные шаблоны

- Количество шаблонных параметров может быть любым
- Шаблонными аргументами могут быть не только типы, но и значения
- Темплейт тоже может быть шаблонным аргументом

```
template<typename T, typename K, uint32_t N>
struct UniversalMultifieldStruct
{
    T field;
    K secondFiled;
    std::array<T, N> internalArray;
    // тип массива и его длину определим в момент инстанциации переменной этого типа
};
```

```
UniversalMultifieldStruct<uint32_t, float, 4> structure = {10, 2.f, {}};
// поле field = имеет тип uint32_t;
// поле secondFiled - float;
// internalArray - массив uint32_t из 4 элементов
```



Прайм

Частичная спецификация шаблона

```
template<typename T, typename K, uint32_t N> struct UniversalMultifieldStruct;
```

- Вы хотите чтобы при определенных параметрах шаблон формировался особенным образом

```
template<typename K, uint32_t N>  
struct UniversalMultifieldStruct<uint32_t, K, N>  
{  
    uint32_t field;  
    K secondFiled;  
    std::array<uint8_t, 122> internalArray;  
};
```

```
UniversalMultifieldStruct<uint32_t, float, 4> structure = {10, 2.f, {}};
```

Аккуратнее со спецификациями шаблонов, это источник серьезных ошибок



Вопрос

```
UniversalMultifieldStruct defaultStructure;
```

А так можно?

```
template<typename T = int, typename K = std::string, uint32_t N = 5>
struct UniversalMultifieldStruct
{
    T field;
    K secondField;
    std::array<T, N> internalArray;
};
// поле field = имеет тип int;
// поле secondField - std::string;
// internalArray - массив int из 5 элементов
```

Кибертрон

Библиотека STL содержит огромное количество полезных шаблонных примитивов.

```
std::array<int, 256> staticArray;    // статический массив целых длиной 256 элементов
std::vector<int> iVector;            // динамический массив целых
std::vector<float> fVector;          // динамический массив чисел с плавающей точкой
std::list<double> list;              // двусвязный список из чисел с плавающей точкой
std::set<std::string> set;           // множество строк
std::map<std::string, int> dictionary; // словарь, поиск числа по строке
```



Продолжение
следует

